LangGraph Integration in Seismic Command: Stateful Graph-Based AI Orchestration for Earthquake Risk Analysis

Yunus Emre Balci
Akdeniz University — Computer Engineering

April 19, 2026

Stack: LangGraph 0.2 + LangSmith + Groq (LLaMA-3.3-70B) + FastAPI + Angular

Contents

1 Introduction **3**

2 Purpose of LangGraph in This Project **3**

3 System Architecture **3**
3.1 Technology Stack . . . 4
3.2 LangGraph Service Entry Point . . . 4

4 Graph 1: Chat Graph **5**
4.1 Purpose and Topology . . . 5
4.2 State Definition . . . 5
4.3 Well-Defined Steps (Nodes) . . . 6
4.4 Hybrid Classifier Node . . . 6
4.5 Conditional Edges . . . 6
4.6 Multi-Turn Persistence . . . 7

5 Graph 2: Building Risk Graph **7**
5.1 Purpose and Topology . . . 7
5.2 State Definition . . . 8
5.3 Well-Defined Steps (Nodes) . . . 9
5.4 Parallel Context Collection . . . 9
5.5 Deterministic Score Engine . . . 9
5.6 Score-Based Conditional Routing . . . 9
5.7 Agentic Evaluator Loop . . . 10

6 LangSmith Integration **11**
6.1 Configuration . . . 11
6.2 What LangSmith Traces . . . 11
6.3 Why LangSmith Matters . . . 12

7 CrewAI and LangGraph Coexistence **12**

8 API Endpoints **13**

9 Screenshots **13**

10 Reflection and Conclusion **18**

1 Introduction

Seismic Command is a full-stack earthquake safety platform for Turkey, built with an Angular frontend, a Spring Boot backend, and a Python-based AI layer. This report documents the **LangGraph integration** added to the existing project—which already contained a CrewAI multi-agent pipeline—demonstrating that both frameworks coexist and run concurrently within the same codebase.

LangGraph was chosen specifically for its ability to model *stateful, conditional workflows* as directed graphs. Unlike a simple chain of LLM calls, LangGraph allows branching, feedback loops, and parallel fan-out, which are exactly the patterns required for dynamic earthquake risk assessment.

Design Rationale

The LangGraph layer exposes **two primary user-facing graphs**:

- **Chat Graph** — A stateful multi-turn conversational assistant that classifies user intent and routes to the appropriate data-fetching or answering pipeline.
- **Building Risk Graph** — An agentic evaluation loop that scores a building's earthquake risk and uses an LLM evaluator to refine its own output before returning a final result.

Both graphs run as async FastAPI endpoints on port 8002 and are consumed by the same existing Spring Boot + Angular application that already integrates CrewAI.

Key Insight

This submission focuses on the two LangGraph workflows that are already surfaced in the product UI and easiest to discuss in class: `/graph/chat` and `/graph/building-risk`. The repository may contain additional experimental graph modules, but they are outside the core scope of this report.

Git repository: https://github.com/YunusEmre3757/advenced_web

2 Purpose of LangGraph in This Project

LangGraph solves three specific problems that a plain LLM call cannot:

1. **Intent routing at runtime.** A user's question about "last week's earthquakes" requires a different pipeline than a question about "is my building safe." LangGraph's conditional edges make this routing explicit and inspectable.

2. **Multi-turn memory.** The Chat Graph uses LangGraph's built-in `add_messages` reducer with a PostgreSQL checkpointer, so conversation history persists across HTTP requests without any manual state management.

3. **Agentic self-correction.** The Building Risk Graph implements an evaluator-worker feedback loop: if the LLM-generated risk summary is too short or misses key factors, the evaluator node injects feedback and the graph re-enters the analysis branch (up to 2 retries). This pattern cannot be expressed as a simple chain.

3 System Architecture

The overall system has three layers. The Angular frontend communicates with the Spring Boot backend (Java), which in turn calls the LangGraph service (Python/FastAPI) for AI-intensive

operations.

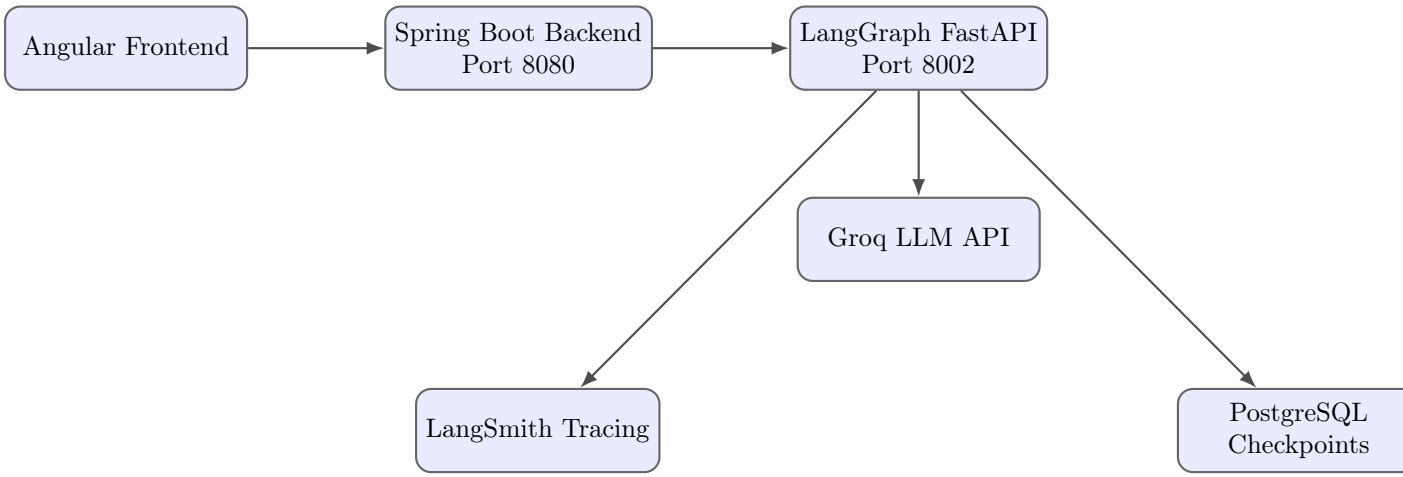

Figure 1: High-level system architecture for the LangGraph integration. The frontend calls Spring Boot, which proxies AI-intensive requests to the LangGraph service.

3.1 Technology Stack

Component	Technology	Purpose
LLM provider	Groq — LLaMA-3.3-70B-Versatile	Graph node LLM calls
Graph framework	LangGraph 0.2	Stateful graph orchestration
Tracing	LangSmith 0.3	Run monitoring and debugging
Structured output	Pydantic v2	Typed LLM responses
API layer	FastAPI + uvicorn	Async HTTP endpoints
Persistence	PostgreSQL + langgraph-checkpoint	Multi-turn chat memory
Frontend	Angular 18	User interface
Backend	Spring Boot 3	Data layer and routing
Co-existing agent	CrewAI	Parallel multi-agent tasks

Table 1: Technology stack used in the LangGraph integration.

3.2 LangGraph Service Entry Point

The service starts with a FastAPI lifespan that initialises LangSmith tracing and the PostgreSQL checkpointer before any request is served:

```
@asynccontextmanager
async def lifespan(_app: FastAPI):
    if LANGSMITH_TRACING:
        os.environ.setdefault("LANGCHAIN_TRACING_V2", "true")
        os.environ.setdefault("LANGCHAIN_PROJECT", LANGSMITH_PROJECT)

    checkpointer = await setup_checkpointer()
    reset_chat_graph()
    get_chat_graph(checkpointer)   # compile once, reuse across requests
    yield
    await get_spring_client().close()
    await close_checkpointer()
```

4 Graph 1: Chat Graph

4.1 Purpose and Topology

The Chat Graph is a stateful multi-turn assistant. It answers earthquake-related questions in Turkish, using Kandilli Observatory data fetched live from the Spring Boot backend.

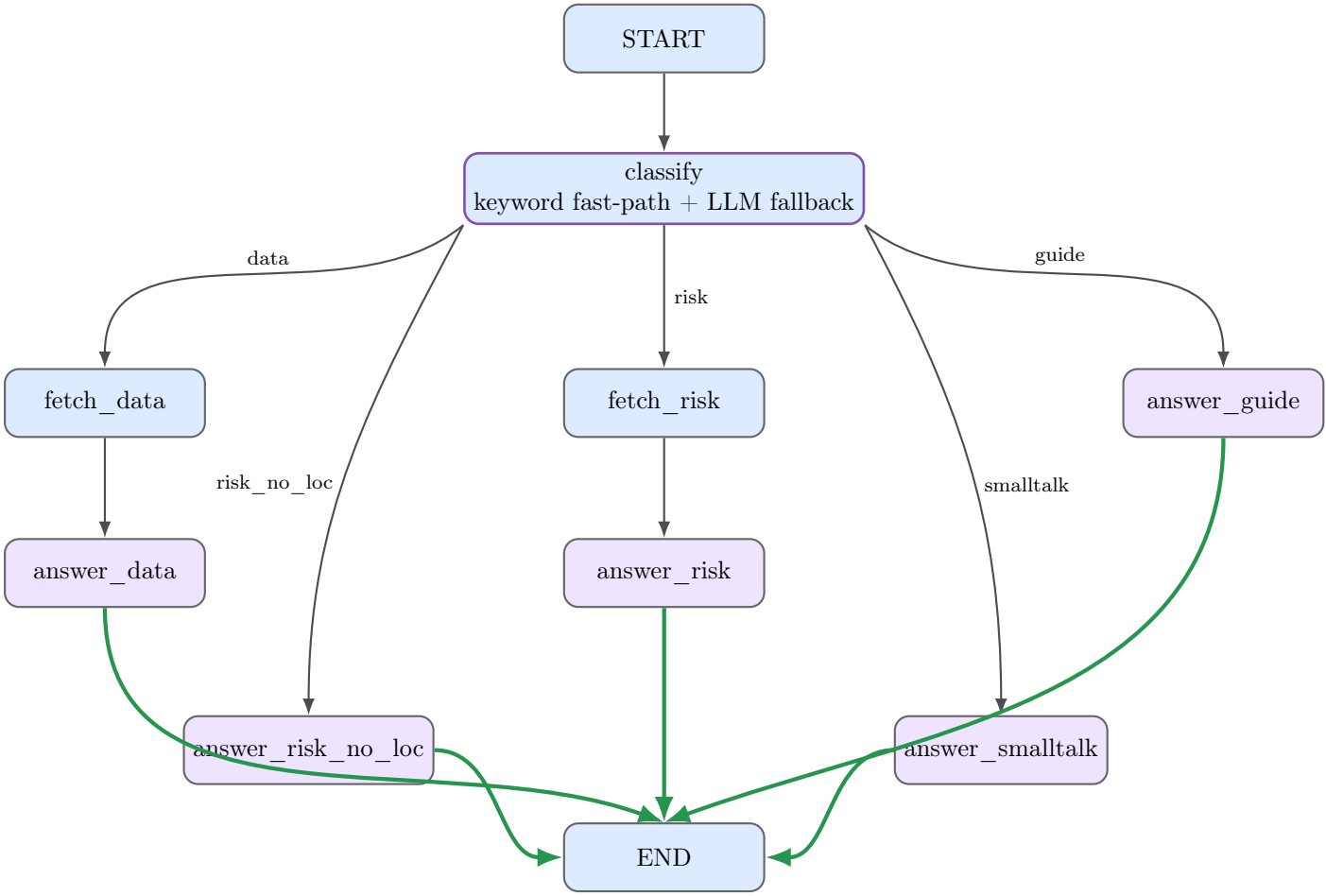

Figure 2: Chat Graph topology. Blue boxes are deterministic/data nodes, purple boxes are LLM answer nodes, and green arrows show successful completion paths.

Key Insight

Color legend: blue = deterministic/data nodes, purple = LLM generation nodes, orange = evaluator node, green = success path, red = retry loop.

4.2 State Definition

LangGraph's `add_messages` reducer automatically appends new messages to the history, enabling multi-turn conversations without manual list management:

```
class ChatState(TypedDict, total=False):
    question: str           # current user turn input
    messages: Annotated[list[Any], add_messages]  # auto-appended
    user_context: dict     # latitude/longitude from device
    category: Category     # set by classify node
    fetched: dict          # live earthquake data
```

```
    sources: list[str]     # citation list
    answer: str            # final assistant reply for the HTTP response
```

4.3 Well-Defined Steps (Nodes)

Each node has a single, clearly named responsibility:

Node	Type	Responsibility
`classify`	Hybrid (keyword + LLM)	Assign category to user message
`fetch_data`	I/O	Pull last 24h earthquakes from Kandilli
`fetch_risk`	I/O + geo computation	Filter nearby quakes via Haversine
`answer_data`	LLM	Respond about recent earthquake data
`answer_guide`	LLM	Provide AFAD safety guide
`answer_risk`	LLM	Assess regional seismic risk
`answer_risk_no_loc`	LLM	Ask for location if missing
`answer_smalltalk`	LLM	General conversation

Table 2: Chat Graph nodes and their single responsibilities.

4.4 Hybrid Classifier Node

The classify node uses a keyword fast-path to avoid an LLM call for unambiguous questions, falling back to structured LLM output only for ambiguous ones:

```
async def classify_node(state: ChatState) -> ChatState:
    question = _current_question(state)
    has_location = "latitude" in (state.get("user_context") or {})

    # keyword fast-path -- no LLM call for clear cases
    keyword_hit = _keyword_classify(question)
    if keyword_hit or DRY_RUN:
        raw = keyword_hit or "smalltalk"
        category = "risk_no_loc" if raw == "risk" and not has_location else raw
        return {"category": category}

    # LLM fallback with structured output for ambiguous questions
    llm = get_structured_llm(ClassifyOutput, temperature=0.0)
    result: ClassifyOutput = await llm.ainvoke([...])
    return {"category": result.category}
```

4.5 Conditional Edges

The `_route` function drives a `add_conditional_edges` call that maps the `category` value to the correct next node:

```
g.add_conditional_edges(
    "classify",
    _route,       # returns state["category"]
    {
        "data":        "fetch_data",
        "guide":       "answer_guide",
        "risk":        "fetch_risk",
        "risk_no_loc": "answer_risk_no_loc",
        "smalltalk":   "answer_smalltalk",
```

```
    },
)
```

4.6 Multi-Turn Persistence

The graph is compiled with a PostgreSQL checkpointer. Each HTTP request passes a `thread_id` (session ID), and LangGraph automatically loads/saves the message history:

```
result = await graph.ainvoke(
    {"question": req.question, "user_context": req.userContext or {}},
    config={"configurable": {"thread_id": req.sessionId}},
)
```

5 Graph 2: Building Risk Graph

5.1 Purpose and Topology

The Building Risk Graph implements an **agentic evaluation loop**. It evaluates a building's earthquake risk by combining deterministic scoring with LLM-generated natural-language analysis, then uses a second LLM call to judge the quality of the analysis and retry if needed.

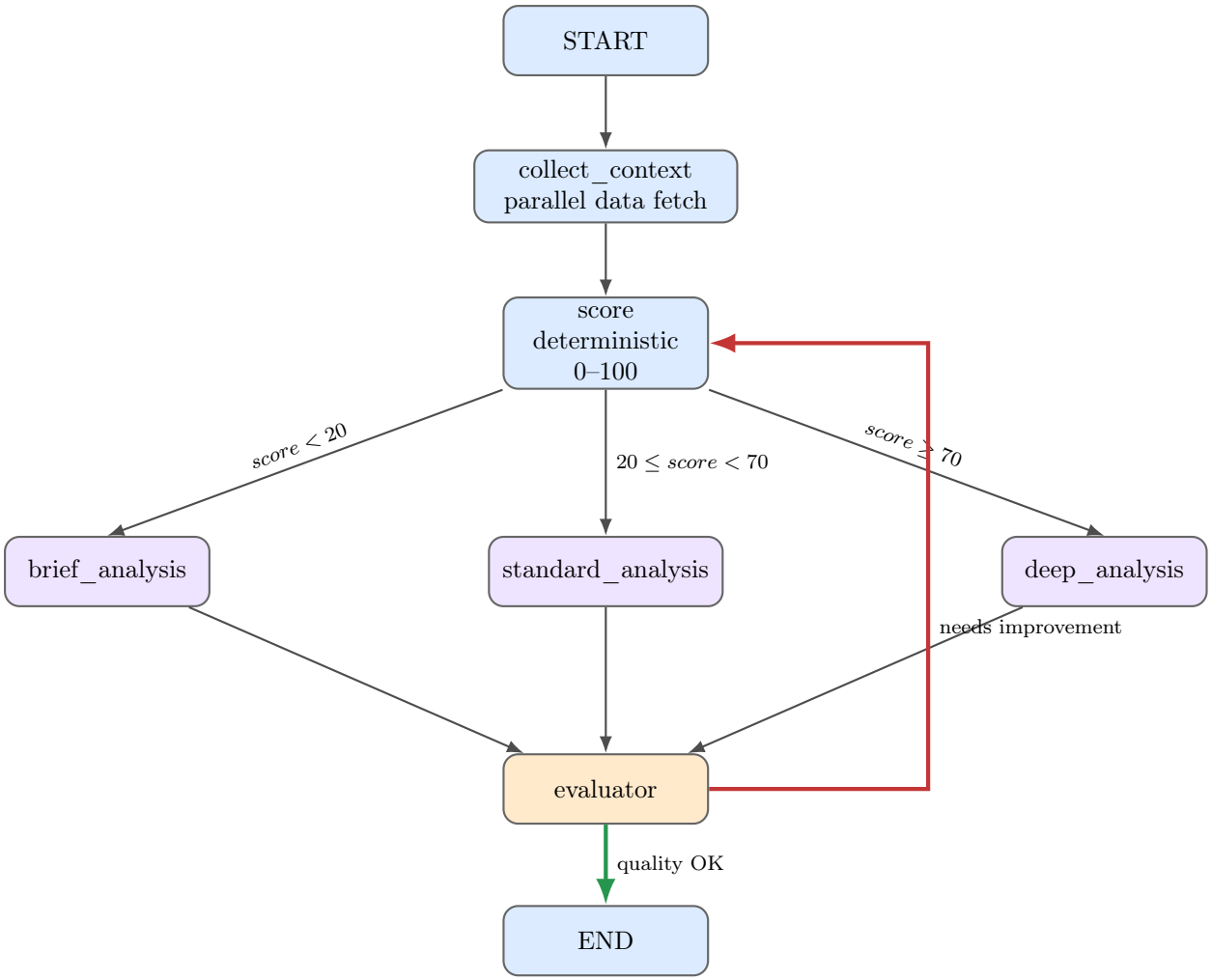

Figure 3: Building Risk Graph topology. Blue boxes are deterministic/data nodes, purple boxes are LLM analysis nodes, the orange box is the evaluator, the green arrow is the success path, and the red arrow is the retry loop.

5.2 State Definition

```
class BuildingRiskState(TypedDict, total=False):
    building: dict[str, Any]          # user-submitted form data
    location: dict[str, Any] | None   # GPS coordinates
    context: dict[str, Any]           # fetched fault/seismic data
    componentScores: dict[str, int]   # per-component scores
    totalScore: int                   # 0-100 aggregate
    level: RiskLevel                  # dusuk/orta/yuksek/kritik
    summary: str                      # LLM-generated narrative
    recommendedActions: list[str]
    evaluator_feedback: str | None    # injected on retry
    retry_count: int
```

5.3 Well-Defined Steps (Nodes)

Node	Type	Responsibility
`collect_context`	I/O (parallel)	Fetch fault lines, historical & recent quakes
`score`	Deterministic	Rule-based scoring across 5 components
`brief_analysis`	LLM (structured)	2-3 sentence summary for low-risk buildings
`standard_analysis`	LLM (structured)	2-paragraph analysis for medium risk
`deep_analysis`	LLM (structured)	3-paragraph detailed analysis for high/critical
`evaluator`	LLM (structured)	Judge quality; inject feedback or approve

Table 3: Building Risk Graph nodes and their single responsibilities.

5.4 Parallel Context Collection

The `collect_context` node uses `asyncio.gather` to fetch three data sources simultaneously, minimising latency:

```
fault_geojson, hist_events, recent_events = await asyncio.gather(
    client.fault_lines(bbox=bbox, simplify=0.008),
    client.historical_events(years=100, min_magnitude=4.5),
    client.recent_earthquakes(hours=168, min_magnitude=3.0, limit=120),
)
```

The node then computes the nearest active fault using a point-to-segment Haversine algorithm and extracts seismic gap signals (years since last M6+ event on the same fault corridor).

5.5 Deterministic Score Engine

The `score` node uses no LLM. It computes five components with fixed rules:

Component	Key Factors	Max Score
Structural	Construction year, floors, system, retrofit	35
Soil	Zone class ZA–ZF	15
Fault Proximity	Distance, slip rate (mm/yr), seismic gap	20
Historical Seismicity	Nearby M4.5+ count, max magnitude	15
Observed Damage	Column cracks, past damage	20
Total		**100**

Table 4: Scoring components in the deterministic rule engine.

```
def _risk_level(total: int) -> tuple[RiskLevel, str]:
    if total < 20:  return "dusuk",  "Dusuk risk"
    if total < 45:  return "orta",   "Orta risk"
    if total < 70:  return "yuksek", "Yuksek risk"
    return "kritik", "Kritik risk"
```

5.6 Score-Based Conditional Routing

After scoring, a conditional edge selects the analysis depth:

```
def _route_by_score(state: BuildingRiskState) -> str:
    total = state.get("totalScore", 0)
```

```
    if total < 20:   return "brief_analysis"
    if total < 70:   return "standard_analysis"
    return "deep_analysis"

g.add_conditional_edges("score", _route_by_score, {
    "brief_analysis":    "brief_analysis",
    "standard_analysis": "standard_analysis",
    "deep_analysis":     "deep_analysis",
})
```

5.7 Agentic Evaluator Loop

All three analysis nodes feed into `evaluator`. This node uses a **second LLM call** to judge the generated summary against four quality criteria. If the quality is insufficient and the retry budget remains, it injects feedback into the state and routes back to `score`, re-entering the full analysis pipeline:

```
async def evaluator_node(state: BuildingRiskState) -> BuildingRiskState:
    # Fast structural checks before calling LLM (saves tokens)
    min_words = {"dusuk": 15, "orta": 25, "yuksek": 35, "kritik": 35}
    if len(summary.split()) < min_words.get(level, 20):
        return {
            "evaluator_feedback": "Ozet cok kisa. Daha ayrintili yaz.",
            "retry_count": retry_count + 1,
        }

    # Full LLM evaluation
    llm = get_structured_llm(EvaluatorOutput)
    result: EvaluatorOutput = await llm.ainvoke([...])

    if result.quality_ok:
        return {"evaluator_feedback": None}         # approve -> END

    return {
        "evaluator_feedback": result.feedback,      # inject feedback
        "retry_count": retry_count + 1,             # retry -> score
    }

def _route_evaluator(state: BuildingRiskState) -> str:
    if state.get("evaluator_feedback") and state.get("retry_count", 0) <
    MAX_RETRIES:
        return "score"    # re-enter pipeline with feedback
    return END
```

Key Insight

The key insight is that graph state does more than store data: it also carries corrective feedback between nodes. When the evaluator rejects a weak answer, its feedback is written into `evaluator_feedback`, and the next analysis attempt reads that field and revises the prompt. This creates a built-in self-correcting loop without any external orchestration code.

6 LangSmith Integration

6.1 Configuration

LangSmith tracing is fully configured and active. Three environment variables are set in `graph/.env`:

```
LANGCHAIN_TRACING_V2=true
LANGCHAIN_PROJECT=seismic-command
LANGCHAIN_API_KEY=lsv2_pt_...
```

The `config.py` module reads these at startup:

```
LANGSMITH_TRACING = os.environ.get("LANGCHAIN_TRACING_V2", "false").lower() == "
    true"
LANGSMITH_PROJECT = os.environ.get("LANGCHAIN_PROJECT", "seismic-command")
```

6.2 What LangSmith Traces

Because LangGraph natively integrates with LangSmith, every run automatically records:

- The full graph execution trace with each node's input and output
- Every LLM call with prompt, response, latency, and token count
- The evaluator feedback loop iterations (visible as nested spans)
- Classification decisions (keyword vs. LLM path)

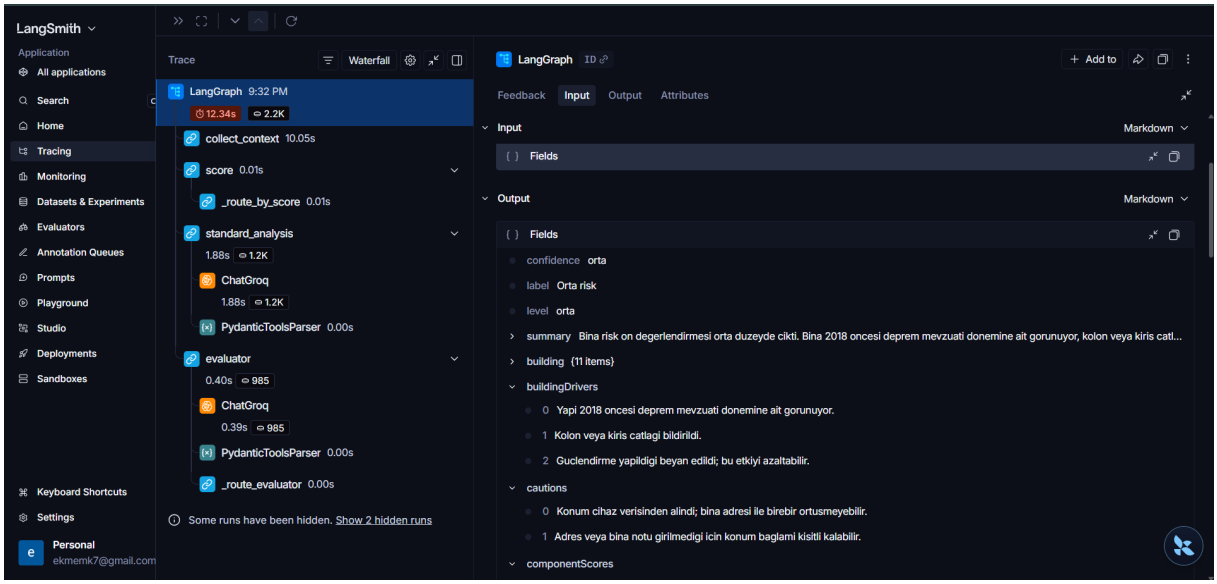

Figure 4: LangSmith trace view for a single Building Risk Graph run. The left panel shows the execution path through `collect_context`, `score`, `_route_by_score`, `standard_analysis`, and `evaluator`. The right panel exposes the structured result returned by the graph, including the risk label, confidence value, summary, driver lists, warnings, and component scores. This screenshot demonstrates that each node in the workflow can be inspected individually, which makes the graph both debuggable and explainable.

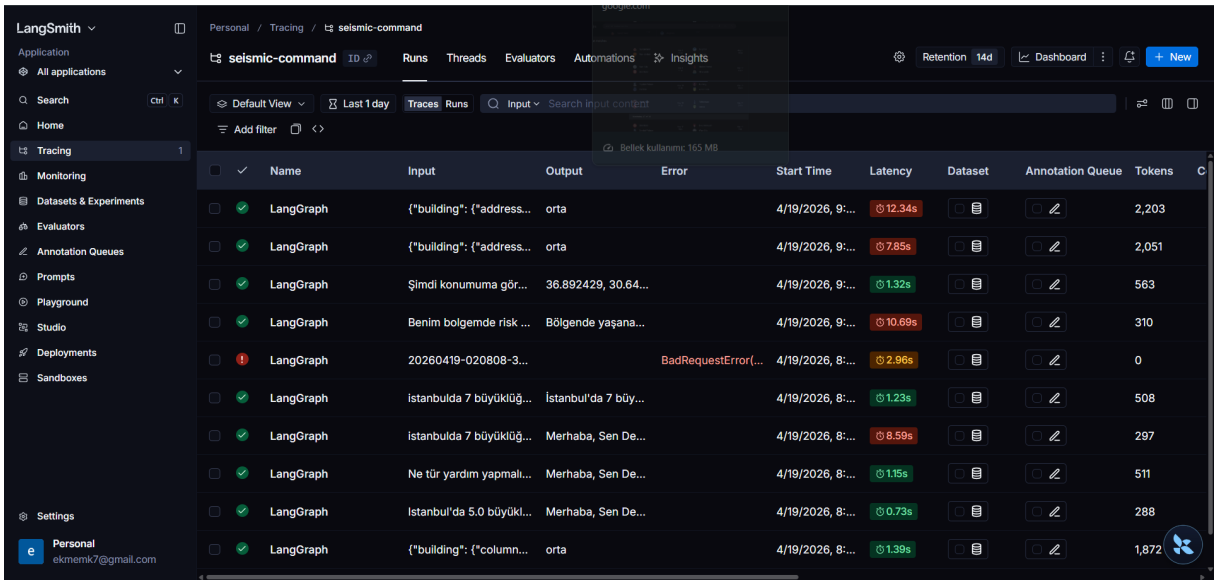

Figure 5: LangSmith project-level run list for the `seismic-command` project. This interface shows that both chat runs and building-risk runs are traced under the same project, together with timestamps, latency, token usage, and error status. In the report, this screenshot serves as evidence that LangSmith integration is active during real application usage rather than isolated development tests.

6.3 Why LangSmith Matters

Design Rationale

Without LangSmith, debugging the evaluator retry loop requires reading log files and manually correlating LLM calls. With LangSmith, each retry appears as a nested span showing exactly which feedback was injected, which analysis node re-ran, and whether the second attempt passed evaluation. This visibility is essential during development and demonstrates that the agentic loop is functioning as designed.

7 CrewAI and LangGraph Coexistence

The assignment explicitly requires that LangGraph be added to an **existing project alongside CrewAI**, with both working simultaneously. This is achieved through service isolation:

- **CrewAI** remains available as its own Python service on port `8001`, focused on multi-agent research and report generation.
- **LangGraph** runs as a separate FastAPI service on port `8002`, focused on stateful chat and agentic building-risk evaluation.
- **Spring Boot** stays in the middle and proxies both AI layers through its existing REST surface, so the Angular frontend can keep using one backend entry point.
- The two frameworks therefore coexist in the same repository and product without runtime conflict: different services, different endpoints, shared application domain.

Integration Status

Both frameworks are active in the same project. LangGraph endpoints are started with `uvicorn seismic_graph.api:app -port 8002`, CrewAI is started from its own service on port `8001`, and the Angular frontend reaches both through the Spring Boot proxy layer.

8 API Endpoints

Endpoint	Graph	Description
`POST /graph/chat`	Chat Graph	Multi-turn stateful chat
`GET /graph/chat/stream`	Chat Graph	SSE token streaming
`POST /graph/building-risk`	Building Risk Graph	Agentic risk assessment
`GET /health`	—	Service health check

Table 5: FastAPI endpoints that form the main LangGraph submission scope.

Key Insight

These endpoints are enough to demonstrate the assignment requirements: clearly defined graph steps, conditional routing, persistent state, and an evaluator loop. They are also the two flows already connected to visible frontend screens, which makes the implementation easy to demo live.

9 Screenshots

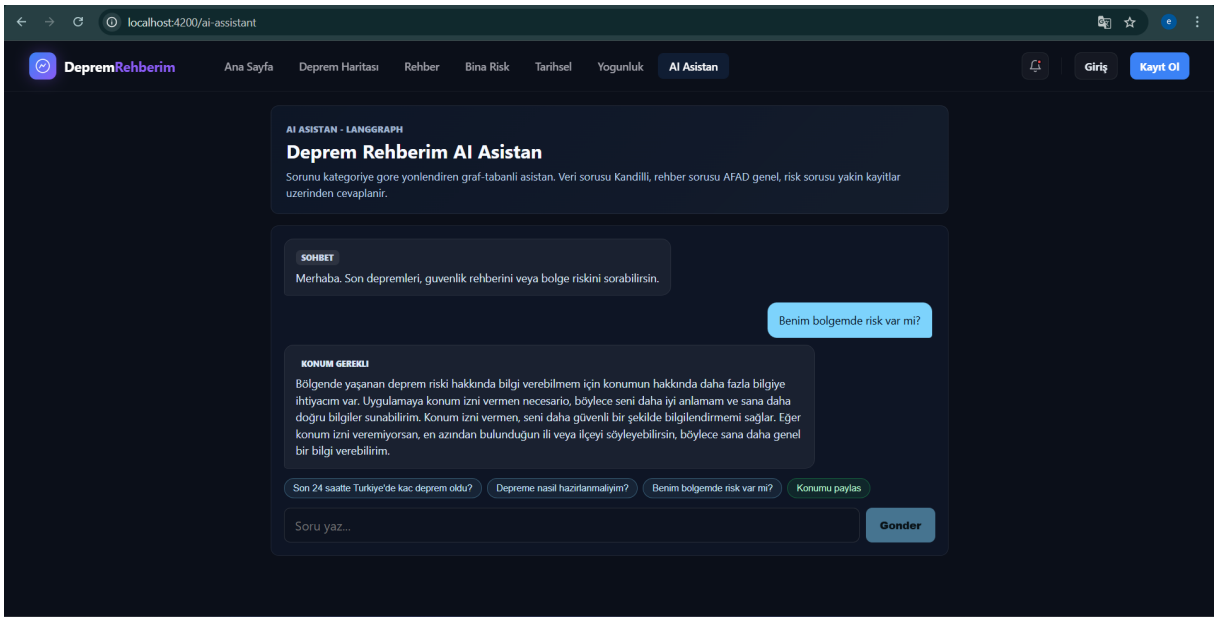

Figure 6: AI Assistant without location permission. The user asks a location-dependent risk question, and the Chat Graph routes the request to the `risk_no_loc` branch because the system does not yet have the user's coordinates. Instead of generating a potentially misleading region-specific answer, the assistant explicitly asks for location context. This screenshot demonstrates conditional routing based on missing state.

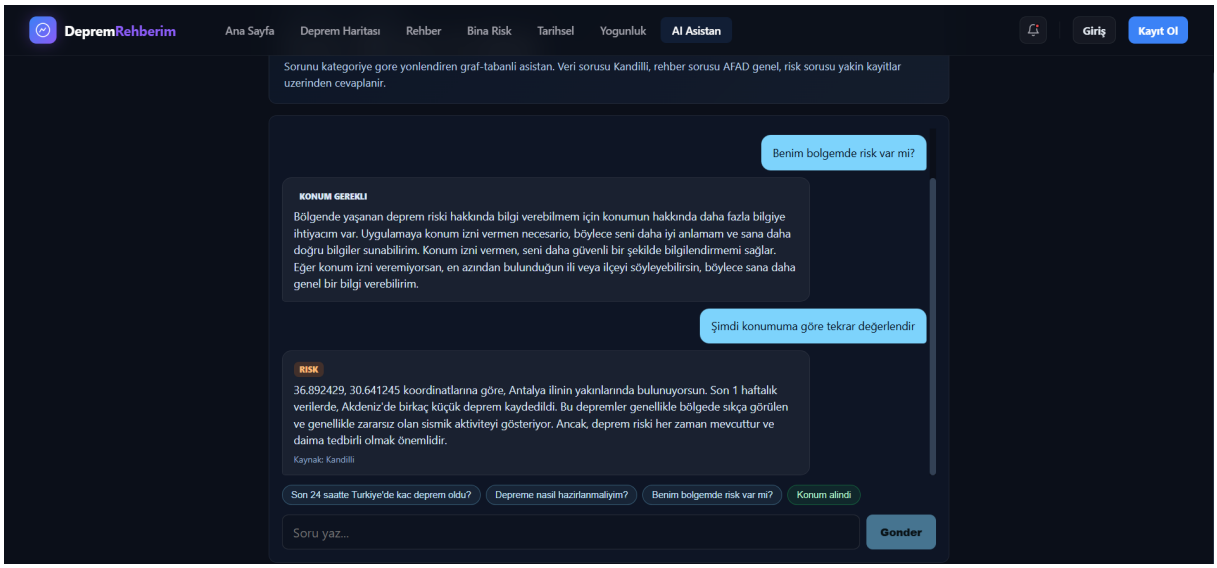

Figure 7: AI Assistant after location sharing in the same session. The earlier `risk_no_loc` response remains visible in the conversation history, and the follow-up request is now answered through the normal risk-analysis path using the newly available coordinates. The screenshot demonstrates multi-turn state persistence, category re-routing, and source attribution in a single conversational flow.

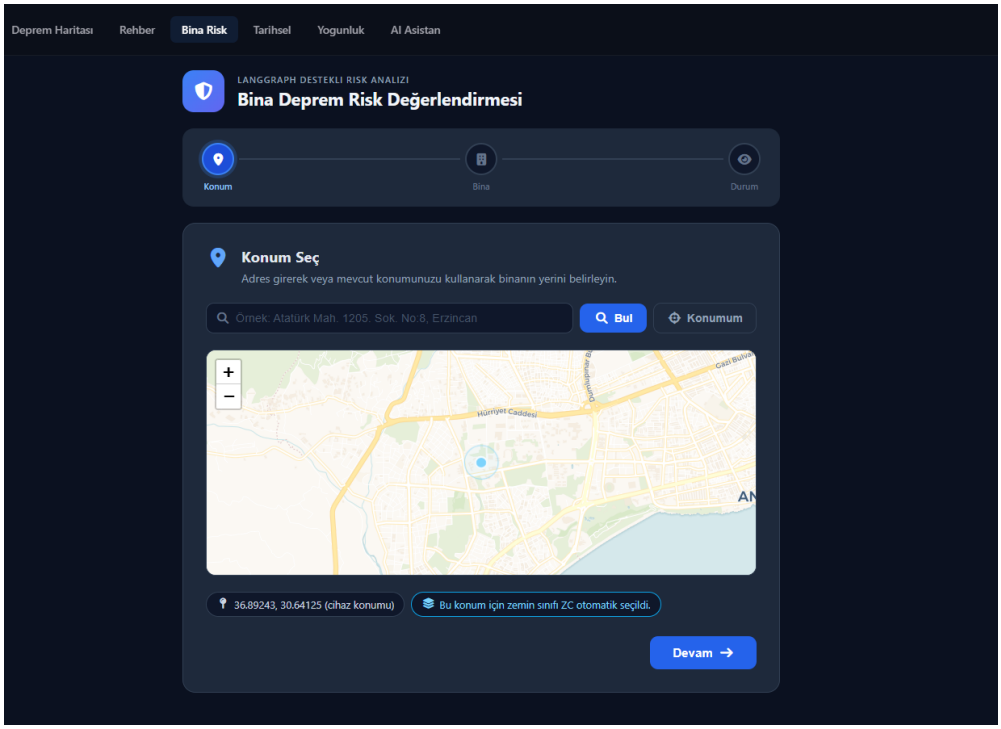

Figure 8: Building Risk workflow, step 1: location selection. The user chooses the building location on the map, and the application automatically derives contextual data such as approximate coordinates and the soil class at that point. This visual matters because the graph uses not only typed building attributes, but also geospatial context gathered before scoring begins.

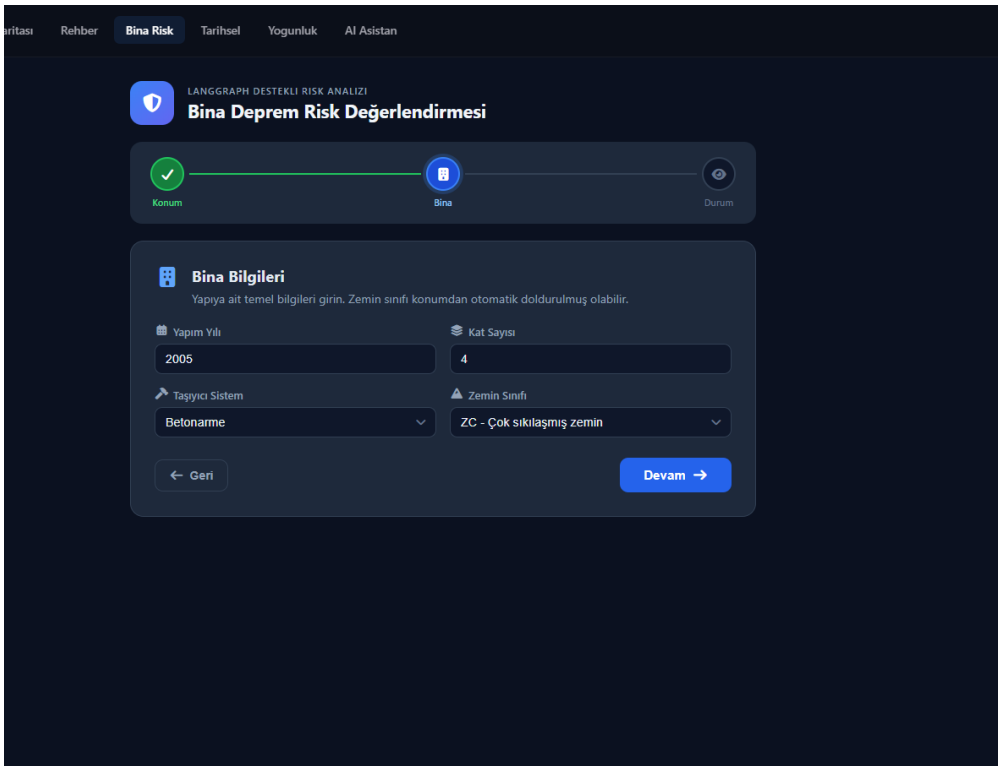

Figure 9: Building Risk workflow, step 2: structural data entry. The user provides the construction year, floor count, structural system, and soil type of the building. These values form the deterministic input for the structural and soil components of the risk score, so this screenshot documents the frontend-to-graph data contract for the main assessment workflow.

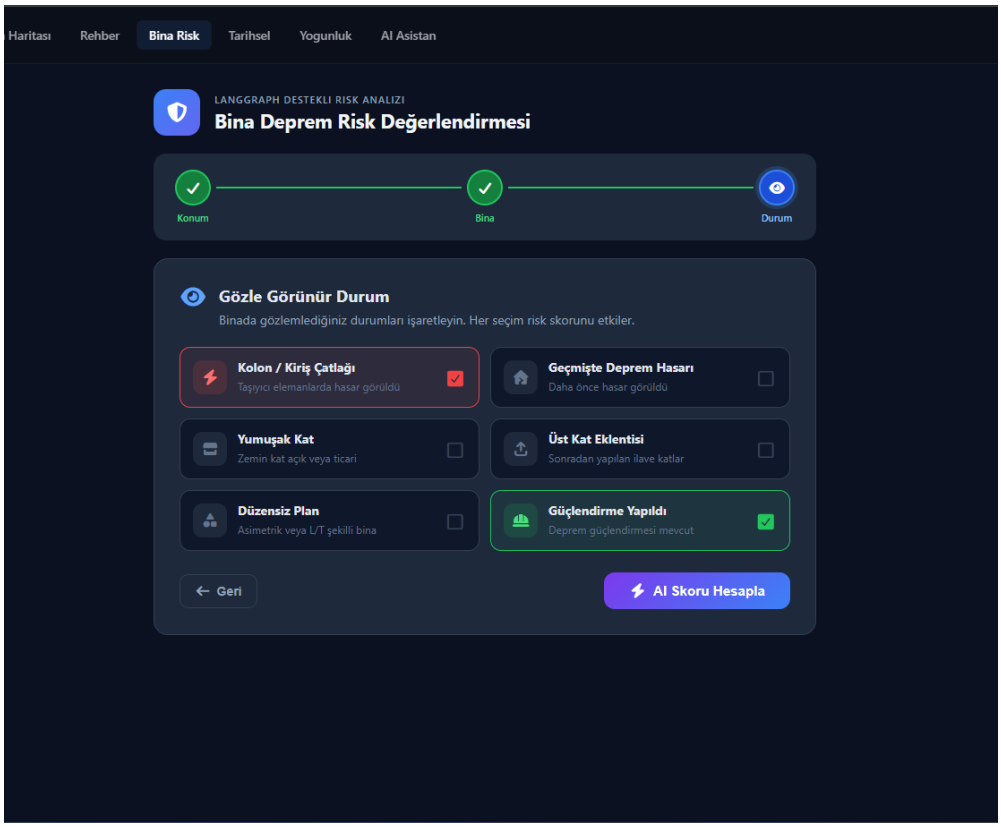

Figure 10: Building Risk workflow, step 3: observed-condition selection. The user marks visible conditions such as cracks, soft-storey risk, past damage, irregularity, and retrofit status. Inside the graph, these selections are translated into deterministic score contributions before the natural-language explanation is generated. This screenshot shows that the final result is grounded in explicit evidence rather than unconstrained prompting alone.

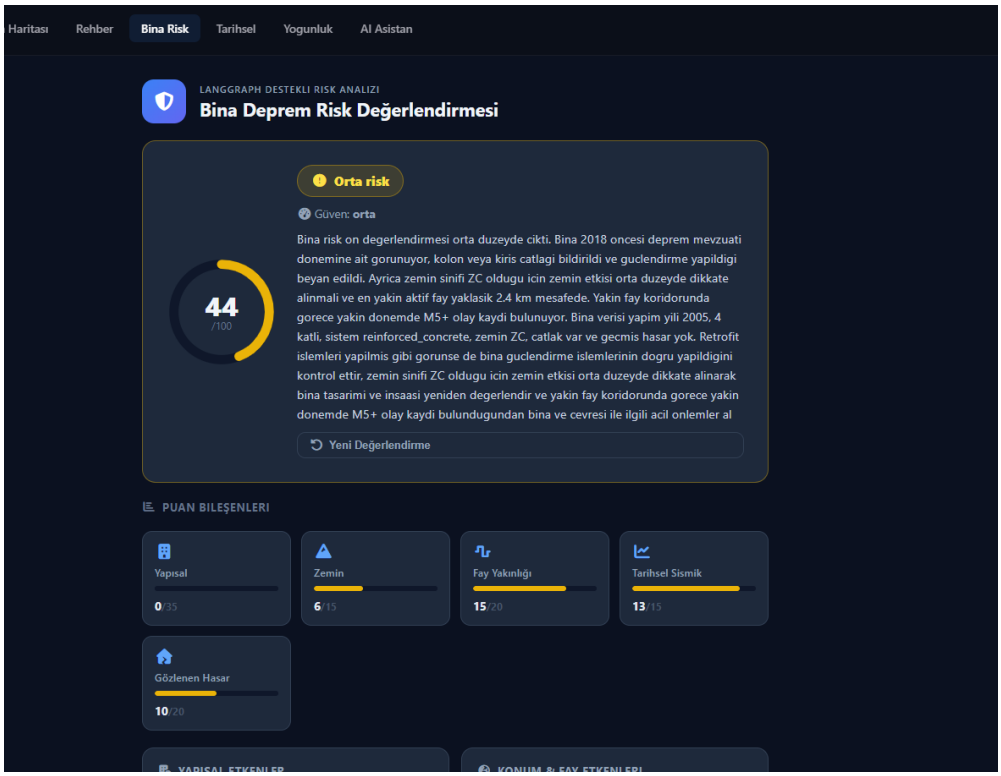

Figure 11: Building Risk result overview. After graph execution, the interface presents the total score, the interpreted risk level, a confidence indicator, and a Turkish summary that combines deterministic scoring with LLM-based explanation. This screenshot is one of the strongest pieces of evidence in the report because it shows the transformation from structured inputs into a user-facing decision output.

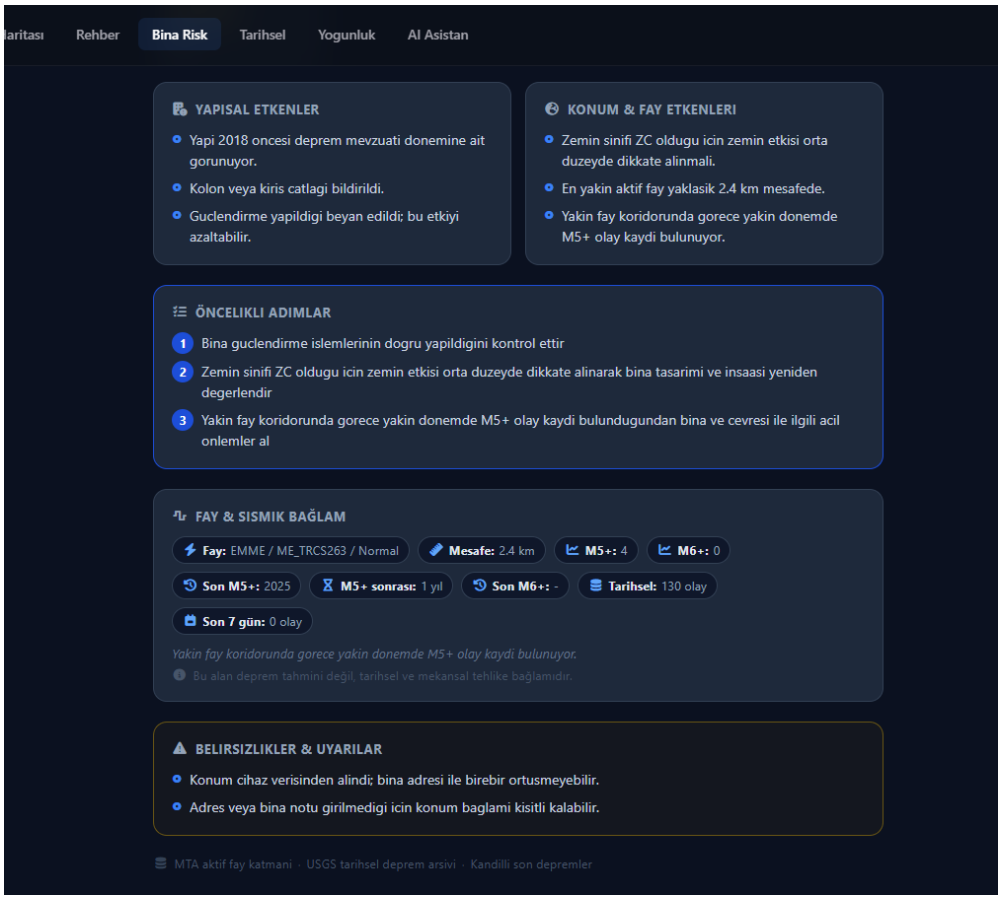

Figure 12: Building Risk result details. Beyond the headline score, the interface exposes structural drivers, location and fault-related drivers, recommended priority actions, fault-corridor metadata, historical seismic context, and uncertainty notes returned by the graph. This detailed screen is valuable because it shows that the system outputs not only a label such as "medium risk," but also interpretable supporting evidence and actionable recommendations.

10 Reflection and Conclusion

Reflection

Graph topology makes intent explicit. Expressing the logic as a directed graph with named nodes and conditional edges—rather than nested if-else blocks—makes the system's decision flow immediately readable. Adding a new question category to the Chat Graph is a matter of adding one node and one edge entry, not refactoring a function.

Reflection

Separating scoring from analysis was the right call. The deterministic score engine in the Building Risk Graph has no LLM dependency, which means it runs in under 1ms, is fully testable, and is never affected by model randomness. The LLM layer only handles the parts that genuinely require language: generating the natural-language summary and evaluating its quality.

Reflection

The evaluator loop has real value. In testing, the brief-analysis node occasionally produced summaries that were too short or omitted location factors. The evaluator catches these cases automatically and requests a more detailed retry. Without the loop, this quality check would have to be done manually or skipped entirely.

Reflection

LangSmith removes debugging guesswork. The evaluator retry loop involves 3-4 LLM calls per request on a bad run. Without tracing, diagnosing why a summary was rejected requires adding print statements and re-running manually. LangSmith shows the full trace—including which feedback was injected and whether the second attempt passed—in a single browser tab.

In summary, LangGraph was integrated into the existing Seismic Command project to add stateful, graph-based AI orchestration alongside the pre-existing CrewAI agents. The two primary graphs—Chat and Building Risk—demonstrate the core LangGraph patterns required by the assignment: well-defined node steps, conditional routing, state management, and an agentic feedback loop, all observable through LangSmith tracing.